

系统开发工具基础

第 1 周实验报告

授课教师 周小伟

姓 名 居里

学 号 25060021025

实验日期 2026 年 8 月 24 日

授课内容 Shell 入门 • Git • LaTeX

GitHub 仓库 https://github.com/oranlj/sdt_course

2026 年 9 月 1 日

目录

1 仓库与提交记录

GitHub 仓库: https://github.com/oranlj/sdt_course

```
MINGW64/d/study/系统开发工具基础/sdt_course
commit 0f43744d8a7613b8ff19c7ad834341fe7d2e5759 (HEAD -> main, origin/main, origin/HEAD)
Author: remus <remus@example.com>
Date: Tue Sep 1 23:19:56 2026 +0800

    更新课后练习

commit 438030a24c2e697ff0f79b2e57937df5e71ba528
Author: remus <remus@example.com>
Date: Tue Sep 1 21:27:14 2026 +0800

    将q01-q04工作文件夹存入

commit 2b81abd86f5cb1cdd2218592b47796e07f32af23
Author: oranlj <mr_juli@163.com>
Date: Tue Sep 1 10:26:41 2026 +0800

    添加q02内容

commit 5d781622ba7be5b3bd03d33260150ea54167cefe
Author: oranlj <mr_juli@163.com>
Date: Tue Sep 1 10:24:28 2026 +0800

: ...skipping...
commit 0f43744d8a7613b8ff19c7ad834341fe7d2e5759 (HEAD -> main, origin/main, origin/HEAD)
Author: remus <remus@example.com>
Date: Tue Sep 1 23:19:56 2026 +0800

    更新课后练习

commit 438030a24c2e697ff0f79b2e57937df5e71ba528
Author: remus <remus@example.com>
Date: Tue Sep 1 21:27:14 2026 +0800

    将q01-q04工作文件夹存入

commit 2b81abd86f5cb1cdd2218592b47796e07f32af23
Author: oranlj <mr_juli@163.com>
Date: Tue Sep 1 10:26:41 2026 +0800

    添加q02内容

commit 5d781622ba7be5b3bd03d33260150ea54167cefe
Author: oranlj <mr_juli@163.com>
Date: Tue Sep 1 10:24:28 2026 +0800

    添加q04内容

commit ace6f4d66996b2df3923c6d781ed808a08be31b0
Author: oranlj <mr_juli@163.com>
Date: Mon Aug 31 16:30:51 2026 +0800

    添加q03命令集,q04相关内容

commit 4690a70f4570c2d154c9758795458997d1378460
Author: oranlj <mr_juli@163.com>
Date: Mon Aug 31 15:12:35 2026 +0800

    添加q01需用指令

commit 7d1c48c4fe3ee38f15005464ff17fc5a06c7c629
Author: oranlj <mr_juli@163.com>
Date: Mon Aug 31 14:04:18 2026 +0800

    添加readme文件
```

图 1: Git 提交记录截图

2 课上检查题

2.1 第 1 题: 含空格文件名的批量整理

主题: Shell 基础与文件系统

题目要求 新建 `q01`, 创建指定目录与文件 (含空格文件名和隐藏文件); 显示绝对路径并长格式列出全部项目; 将所有 `.txt` 复制到 `work/< 学号 >/` 保留目录结构; 设置目

录权限 750、文件权限 640；生成 `inventory.txt`。

关键命令

```
1 # 创建 q01 目录并进入
2 mkdir -p q01/input/docs q01/input/tmp
3 cd q01
4 # 创建 note one.txt .secret.txt empty.txt
5 echo -e "alpha\nbeta" > "input/docs/notes one.txt"
6 echo "hidden" > input/docs/.secret.txt
7 touch input/tmp/empty.txt
8 # 创建日志文件
9 cat > input/run.log << 'EOF'
10 [2026-08-26 09:00:00] INFO: service initialized
11 [2026-08-26 09:00:01] INFO: input directory ready
12 EOF
13 # 显示绝对路径，所有（含隐藏）文件
14 pwd
15 ls -laR input
16
17 # 创建目标目录
18 mkdir -p "work/25060021025"
19
20 # 进入 input 目录，复制所有 txt 文件并保留相对目录结构
21 cd input
22 find . -type f -name "*.txt" -exec cp --parents {} "../work/25060021025/"
23 \;
24 # 返回 q01 根目录
25 cd ..
26
27 # 设置权限
28 find "work/25060021025" -type d -exec chmod 750 {} \;
29 find "work/25060021025" -type f -exec chmod 640 {} \;
30 # 生成 inventory
31 find "work/25060021025" -type f -printf "%P %s\n" > inventory.txt
```

Listing 1: q01 操作命令

运行结果

```
/home/remus/system_dev_tools/experiments/week01/q01
input:
total 20
drwxrwxr-x 4 remus remus 4096 Aug 31 16:49 .
drwxrwxr-x 3 remus remus 4096 Aug 31 16:49 ..
drwxrwxr-x 2 remus remus 4096 Aug 31 16:49 docs
-rw-rw-r-- 1 remus remus   98 Aug 31 16:49 run.log
drwxrwxr-x 2 remus remus 4096 Aug 31 16:49 tmp

input/docs:
total 16
drwxrwxr-x 2 remus remus 4096 Aug 31 16:49 .
drwxrwxr-x 4 remus remus 4096 Aug 31 16:49 ..
-rw-rw-r-- 1 remus remus    7 Aug 31 16:49 .secret.txt
-rw-rw-r-- 1 remus remus   11 Aug 31 16:49 'notes one.txt'

input/tmp:
total 8
drwxrwxr-x 2 remus remus 4096 Aug 31 16:49 .
drwxrwxr-x 4 remus remus 4096 Aug 31 16:49 ..
-rw-rw-r-- 1 remus remus    0 Aug 31 16:49 empty.txt
```

图 2: 显示 q01 绝对路径，列出项目

```
docs/.secret.txt 7
docs/notes one.txt 11
tmp/empty.txt 0$
```

图 3: inventory.txt 内容

2.2 第 2 题：随机访问日志统计

主题: Shell 管道与文本处理

题目要求 编写 `analyze.sh` 接收 CSV 路径；文件不存在时写 `stderr` 并非零退出；用管道 + `awk` + `sort` + `head` 输出 5xx 最多的前 2 个 path（次数降序、path 字典序）；计算

平均 latency 保留两位小数；分别对存在和不存在的文件运行。

脚本代码

```
1 cat > analyze.sh <<'EOF'
2 #!/bin/bash
3
4 # 判断参数数量
5 if [ $# -ne 1 ]; then
6 echo "usage: $0 csvfile" >&2
7 exit 1
8 fi
9
10 CSV="$1"
11
12 # 判断文件是否存在
13 if [ ! -f "${CSV}" ]; then
14 echo "error: file ${CSV} does not exist" >&2
15 exit 2
16 fi
17
18 ##### 任务1: 5xx最多前2个path
19 # 跳过表头, 第4列status, 匹配500-599; 统计每个path频次; 排序: 次数降, path字典升; 取前2
20 awk -F',' 'NR>1 && $4~/^5[0-9][0-9]$/ {print $3}' "${CSV}" \
21 | sort \
22 | uniq -c \
23 | sort -k1,1nr -k2,2 \
24 | head -n 2
25
26 ##### 任务2: 计算latency_ms平均值, 保留两位小数, 跳过表头
27 awk -F',' 'NR>1 {sum += $5; cnt++;} END {printf "%.2f\n", sum/cnt}' "${CSV}"
28 EOF
```

Listing 2: analyze.sh

运行结果

```
remus@remus-VMware-Virtual-Platform:~/system_dev_tools/experiments/_week1/q02$ ./analyze.sh access.csv
3 /api/orders
2 /api/users
172.22
remus@remus-VMware-Virtual-Platform:~/system_dev_tools/experiments/_week1/q02$ ./analyze.sh hello.csv
error: file hello.csv does not exist
```

图 4: q02 运行结果

2.3 第3题：制造、解决并解释一次合并冲突

主题: Version Control and Git

操作步骤

```
1  mkdir q03
2  cd q03
3  git init
4
5  # main初始提交
6  echo "mode=normal" > config.txt
7  git add config.txt
8  git commit -m "initial commit: mode normal"
9
10 # feature-a
11 git checkout -b feature-a
12 echo "mode=safe" > config.txt
13 git add config.txt
14 git commit -m "feature-a: set mode safe"
15
16 # 回到main创建feature-b
17 git checkout main
18 git checkout -b feature-b
19 echo "mode=fast" > config.txt
20 git add config.txt
21 git commit -m "feature-b: set mode fast"
22
23 # 回到main, 合并
24 git checkout main
25 git merge feature-a
26
27 # 合并feature-b产生冲突
28 git merge feature-b
29
30 # 手动解决冲突
31 cat > config.txt <<'EOF'
32 mode=safe
33 note=reviewed
34 EOF
35
36 git add config.txt
37 git commit -m "merge feature-b, resolve conflict: keep safe, add note"
38
39 git status
40 git log --all --graph --decorate --oneline
```

Listing 3: q03 操作命令

冲突解决后的 config.txt

```
1 mode=safe
2 note=reviewed
```

Listing 4: config.txt

运行结果，含 Git 历史截图

```
在 /home/remus/system_dev_tools/experiments/week01/q03/.git/ 初始化空的 Git 仓库
[main (根提交) be2d64c] initial commit: mode normal
 1 file changed, 1 insertion(+)
 create mode 100644 config.txt
切换到一个新分支 'feature-a'
[feature-a 649c5da] feature-a: set mode safe
 1 file changed, 1 insertion(+), 1 deletion(-)
切换到分支 'main'
切换到一个新分支 'feature-b'
[feature-b 9170ee8] feature-b: set mode fast
 1 file changed, 1 insertion(+), 1 deletion(-)
切换到分支 'main'
更新 be2d64c..649c5da
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
自动合并 config.txt
冲突 (内容) : 合并冲突于 config.txt
自动合并失败, 修正冲突然后提交修正的结果。
[main 549d4ee] merge feature-b, resolve conflict: keep safe, add note
位于分支 main
无文件要提交, 工作区干净
* 549d4ee (HEAD -> main) merge feature-b, resolve conflict: keep safe, add note
| \
| * 9170ee8 (feature-b) feature-b: set mode fast
* | 649c5da (feature-a) feature-a: set mode safe
|/
* be2d64c initial commit: mode normal
```

图 5: 运行结果

2.4 第 4 题：修复并构建一页技术说明

主题: LaTeX 文档编辑

report.tex 关键代码

```
1 \documentclass{article}
2 \usepackage{amsmath}
```

```
3 #为了正确渲染中文
4 \usepackage{ctex}
5 \begin{document}
6     \title{Tool Report}
7     \author{姓名-学号}
8     \maketitle
9
10    \section{Result}
11
12    下面给出核心计算公式，见公式~\eqref{eq:energy}:
13    \begin{equation}
14        E = mc^2
15        \label{eq:energy}
16    \end{equation}
17    由公式~\eqref{eq:energy}可以看到质能关系。
18
19    表~\ref{tab:param}展示两组实验参数。
20    \begin{table}[h]
21        \centering
22        \caption{实验参数表}
23        \label{tab:param}
24        \begin{tabular}{|c|c|}
25            \hline
26            参数A & 参数B \\
27            \hline
28            100 & 200 \\
29            \hline
30            300 & 400 \\
31            \hline
32        \end{tabular}
33    \end{table}
34    从表~\ref{tab:param}可以读取两组2行2列的实验数据。
35
36    \end{document}
```

Listing 5: report.tex

生成的 PDF 截图

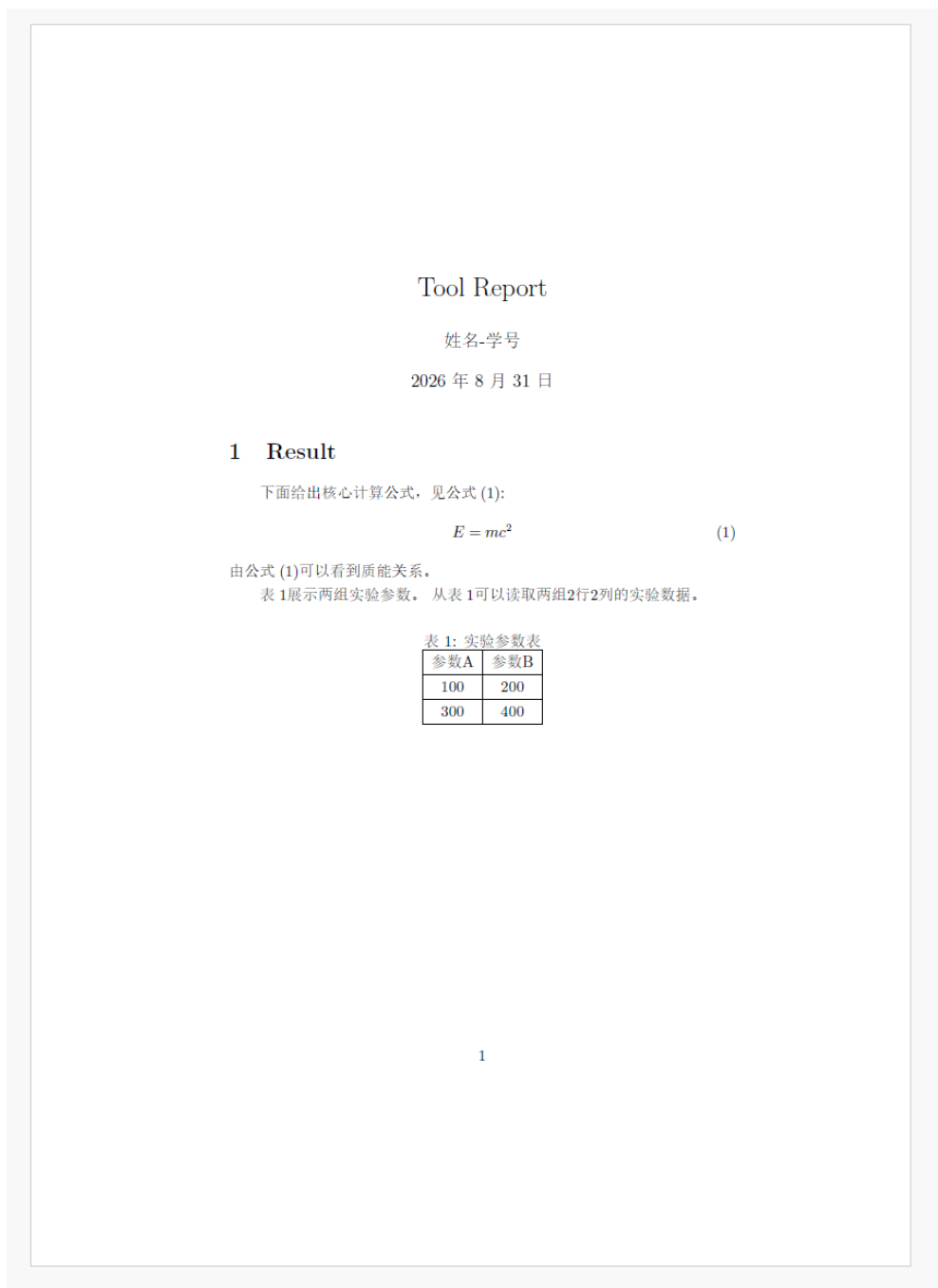


图 6: report.pdf 渲染效果

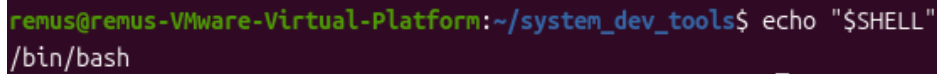
3 课后练习（11 道）

本节题目选自 Missing Semester 2026 的 [Shell 课后练习](#) 与 [Git 课后练习](#)，选取其中相对基础、便于复现的题目，并保留原题编号以便查阅。

3.1 Shell 课后练习

1. 确认 Shell 环境。运行下列命令确认当前使用 Bash 或 Zsh；若输出类似 `/bin/bash` 或 `/usr/bin/zsh`，说明环境符合要求。

```
1 echo "$SHELL"
```

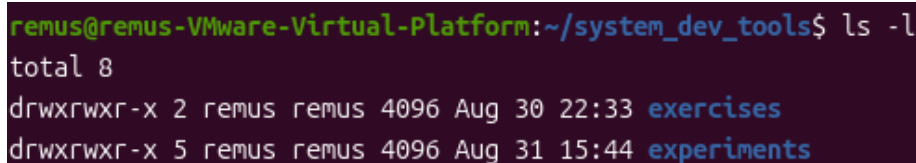


```
remus@remus-VMware-Virtual-Platform:~/system_dev_tools$ echo "$SHELL"
/bin/bash
```

图 7: Shell 环境确认结果

2. `ls` 的长格式输出。运行 `ls -l /`，查阅手册并解释每行开头 10 个字符。第 1 个字符表示文件类型，后 9 个字符依次表示属主、属组和其他用户的读、写、执行权限。

```
1 ls -l /
2 man ls
```

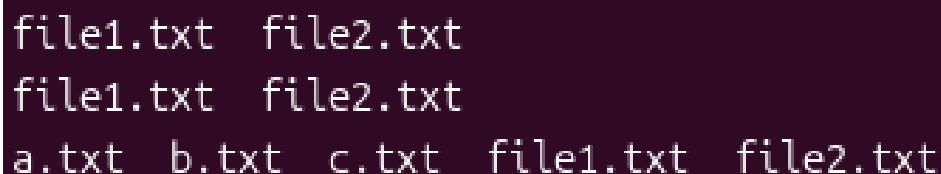


```
remus@remus-VMware-Virtual-Platform:~/system_dev_tools$ ls -l
total 8
drwxrwxr-x 2 remus remus 4096 Aug 30 22:33 exercises
drwxrwxr-x 5 remus remus 4096 Aug 31 15:44 experiments
```

图 8: 长格式文件信息与权限字段

3. `glob` 通配符新建测试目录和若干文件，比较 `*.txt`、`file?.txt` 与花括号展开的匹配结果。

```
1 mkdir -p glob-test && cd glob-test
2 touch file1.txt file2.txt fileA.log
3 ls *.txt
4 ls file?.txt
5 touch {a,b,c}.txt
6 ls *.txt
```



```
file1.txt  file2.txt
file1.txt  file2.txt
a.txt  b.txt  c.txt  file1.txt  file2.txt
```

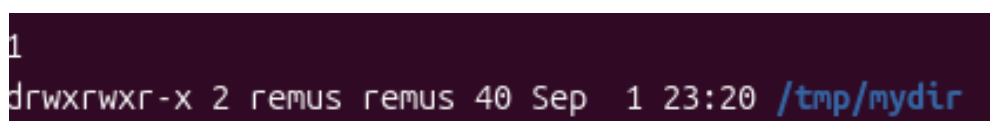
图 9: glob 通配符匹配结果

4. 区分标准输出与标准错误运行同时产生正常输出和错误输出的命令，先分别重定向 stdout 与 stderr，再将二者写入同一文件。

```
1 ls /nonexistent /tmp >stdout.log 2>stderr.log
2 ls /nonexistent /tmp >all.log 2>&1
```

5. 使用退出状态、&& 与 ||。查看上一条命令的退出状态，并写一条仅在 /tmp/mydir 不存在时才创建目录的命令。

```
1 false
2 echo "$?"
3 [ -d /tmp/mydir ] || mkdir -p /tmp/mydir
```

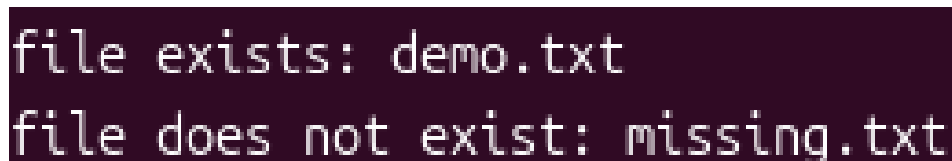


```
1
drwxrwxr-x 2 remus remus 40 Sep  1 23:20 /tmp/mydir
```

图 10: 退出状态与条件执行结果

6. 编写文件存在检查脚本。编写脚本接收文件名参数 \$1，使用 test -f 或方括号判断文件是否存在，并输出不同提示。

```
1 #!/bin/bash
2 if [ -f "$1" ]; then
3     echo "file exists: $1"
4 else
5     echo "file does not exist: $1"
6 fi
```



```
file exists: demo.txt
file does not exist: missing.txt
```

图 11: 文件存在性检查脚本运行结果

3.2 Git 课后练习

7. 通过入门资料理解 Git 数据模型。阅读 Learn Git Branching 入门教程，并尝试把提交、分支和 HEAD 与 Git 的快照、引用模型对应起来。

```
1 git help commit
2 git help branch
3 git log --all --graph --decorate --oneline
```

8. 克隆课程网站仓库并查看历史。克隆课程仓库，将历史显示为图，并查询最后修改 `README.md` 的提交和作者。

```
1 git clone https://github.com/missing-semester/missing-semester.git
2 cd missing-semester
3 git log --all --graph --decorate --oneline
4 git log -1 -- README.md
```

9. 练习 `git stash`。修改仓库中的已有文件，观察保存修改前后的状态和历史，再使用 `git stash pop` 恢复修改，并思考该功能适用的场景。

```
1 printf '\npractice change\n' >> README.md
2 git status
3 git stash
4 git log --all --oneline
5 git stash pop
```

10. 创建历史图别名。在 Git 配置中创建 `graph` 别名，使 `git graph` 等价于显示完整、带装饰的提交图。

```
1 git config --global alias.graph \
2     "log --all --graph --decorate --oneline"
3 git graph
```

11. 配置全局忽略文件。

```
1 git config --global core.excludesfile ~/.gitignore_global
2 printf '.DS_Store\n*.swp\n' > ~/.gitignore_global
3 git config --global --get core.excludesfile
```

4 解题感悟

4.1 课堂检查题体会

q01 让我认识到，shell 操作要正确处理路径、文件名。面对含空格的文件名，需要使用引号或转义；复制文件时不仅要找到目标文件，还要考虑如何保留目录结构；设置权限时则要区分目录与普通文件。

q02 体现了 Shell 管道的思想。日志统计时，筛选、计数、排序和截取分别交给 `awk`、`sort`、`uniq` 和 `head`。我同时认识到编写一个完整的脚本应保证其稳定通用，应检查输入文件、把错误信息写入 `stderr`，用非零退出码明确报告失败。

q03 通过实际制造合并冲突，使我理解了 Git 分支。当两个分支修改同一位置时，Git 无法替使用者判断哪一份内容更合理，因此冲突需要人工理解上下文后解决。同时

我认识到“操作后验证”的重要性。

q04 则让我初步了解了 LaTeX 的结构化写作方式。修改源码后还需要正确编译，才能让编号、目录与引用等保持一致。

4.2 课后练习体会

Shell 课后练习强化了一些基础概念。通过观察 `ls -l` 的权限字段、比较不同 glob 模式、分离 stdout 与 stderr、检查退出状态以及编写文件存在性判断脚本，我拓展了常用命令相关知识。同时我认识到程序的正常输出与错误信息应当分开管理，退出码也可以直接参与后续流程控制。

Git 课后练习则补充了版本控制的日常使用方法。克隆课程仓库并查看提交图，让我能把提交、分支和 HEAD 与 Git 的快照和引用模型联系起来；`git stash` 适合临时保存尚未完成的修改，别名可以简化常用的历史查看命令，全局忽略文件则能避免把编辑器或操作系统生成的临时文件误加入仓库。这些练习让我意识到，Git 不只是“提交代码”的工具。

4.3 总结与改进

本周最大的收获，是初步感悟了如何利用开发工具解决问题。遇到不确定的参数时可以查阅手册；执行操作后进行检查。对于可能覆盖、移动或删除文件的命令，应先在测试目录验证，再用于真实数据；对于 Git，则应养成每次小量提交的习惯，使开发工具得以有效利用。